PRUEBA TÉCNICA PARA DESARROLLADOR PYTHON DE INTEGRACIONES

Edición 2026 · Equipo de Integraciones © Fracttal Tech S.L — Todos los Derechos Reservados

---

Para el desarrollo de esta prueba puede utilizar todas las librerías de Python que crea convenientes, y organizar el proyecto de la forma que crea más adecuada para su fácil entendimiento y ejecución.

Puede —y esperamos que lo haga— apoyarse en asistentes de IA. No evaluamos si los usó, sino qué entendió del problema, qué verificó y qué dejó operable para otra persona. Lo único que descalifica es no declarar ese uso o declararlo falsamente.

Esta prueba no solo evalúa que su código funcione, sino también la claridad y simplicidad de la solución, el criterio al trabajar con datos y con código que no escribió usted, y la **verificabilidad**: que alguien más pueda ejecutarlo y saber si el resultado es correcto.

Duración: 2 días calendario. Trabajo estimado: 5–7 horas. No buscamos que resuelva todo. Preferimos una entrega parcial bien explicada antes que una completa que no pueda sustentar.

En algunos casos el enunciado **no dice qué hacer**. Es deliberado: decida usted, aplique la decisión de forma consistente y explíquela en el reporte. No penalizamos la decisión; penalizamos no haberla visto.

Esta prueba consta de **3 puntos, todos con igual importancia**.

Material adjunto: `mock_erp.py`, `transform_registry.py` y `propuesta_arquitectura.md`.

Punto 1 · Flujo de integración asíncrono contra la API de un ERP

`mock_erp.py` simula el sistema de un cliente. Se levanta con `python3 mock_erp.py` en `http://localhost:8099`. Credenciales: usuario `candidato`, password `prueba2026` (Basic auth en `/auth/token`). Los datos son ficticios.

Método	Ruta	Qué hace
POST	`/auth/token`	Basic auth → `{"access_token", "expires_in"}`
GET	`/api/v1/locations`	Catálogo de ubicaciones válidas
GET	`/api/v1/assets?page=N`	Activos, paginado (20 por página)
GET	`/api/v1/assets/{code}/readings`	Lecturas de medidor de un activo

Implemente un flujo que se ejecute de forma **asíncrona usando una cola de trabajo** (queda a su consideración el framework) y que **procese varios activos concurrentemente**. Debe incluir al menos cuatro etapas:

a. Validación de los datos que devuelve el ERP. Los campos que llegan y los que necesita el destino no son los mismos, y no todos los registros vienen completos. Declare **en un solo lugar** qué forma debe tener un activo válido y aplíquelo a cada registro, en vez de repartir comprobaciones por el flujo. Un registro que no pase queda guardado con el motivo y el proceso continúa: un inválido no puede tumbar el lote.

b. Enriquecimiento: por cada activo válido, consultar sus lecturas (`/assets/{code}/readings`) y quedarse **solo con la última lectura de cada medidor**. Este paso depende del anterior: si un activo se rechazó, no se piden sus lecturas.

c. Normalización y hash: llevar cada activo a este contrato y generar un **hash/ID único** del registro procesado, estable entre corridas.

Campo destino	Tipo	Regla
`code`	texto, único	identificador del activo
`name`	texto, máx. **50** caracteres	si no cabe, se trunca y **se registra**
`active`	booleano	derivado del estado que manda el ERP
`cost`	decimal	dos decimales, sin ambigüedad de separador
`location`	texto	**debe existir** en `/api/v1/locations`; si no, el registro se rechaza
`installed_at`	fecha/hora en UTC	—

d. Persistencia en una base local (SQLite o Postgres en Docker). **No duplicar por reintentos ni por corridas repetidas** — use `code` como clave. Los rechazados se guardan con el motivo.
Al terminar, el proceso imprime un **resumen de la corrida**: leídos, transformados, cargados y rechazados, con motivos.

Notas:

- Este ERP es tan real como los de nuestros clientes. **No asuma que responde bien ni que sus datos están limpios**, y no dé por buena una corrida solo porque terminó sin excepciones.
- Considere el log del procesamiento y la gestión de reintentos ante error.
- **Entregue las evidencias de la ejecución**, no solo el código: el archivo de la base de datos resultante (o un `dump`), el log de una corrida completa y el log de una **segunda corrida seguida**.

Punto 2 · Código en producción que hay que revisar

`transform_registry.py` está **hoy en producción**. Desde la plataforma web el usuario elige qué operación aplicar a los datos antes de enviarlos al ERP, y el equipo agrega operaciones nuevas cada

semana. El módulo cumple el requisito con el que se escribió: **se puede agregar una operación nueva sin modificar el código existente**.
Aun así, no lo dejaríamos como está.

a. Léalo y díganos qué puede salir mal. Ordene lo que encuentre por gravedad y explique, en una línea cada uno, **qué tendría que pasar para que ese problema se manifieste**.

b. Corríjalo, conservando el requisito original (agregar una operación nueva sin tocar el código existente).

c. Escriba **los tests** que demuestren tanto que la corrección funciona como que el requisito original se mantiene.

Nota: este punto no se trata de reescribirlo bonito. Se trata de qué ve y qué prioriza cuando le entregan código ajeno que ya está corriendo.

Punto 3 · Revisión de una propuesta de arquitectura

En Fracttal, la mayoría de nuestros clientes necesita integrar su ERP con la plataforma. Hoy esas integraciones se hacen por *polling* y tenemos tres problemas: consultar todo o por rango de tiempo es ineficiente cuando no hay datos nuevos; los clientes quieren tiempo real y hacer polling cada segundo no es viable; y las soluciones a la medida no escalan, porque cada cliente nuevo consume personal.

Contexto que debe tener en cuenta: Fracttal es un servicio en la nube y **todas las compañías comparten una sola base de datos** (multitenant).

`propuesta_arquitectura.md` es una propuesta de solución generada con un asistente de IA, sin editar. **Su trabajo no es diseñar la arquitectura: es revisar esa propuesta**, como revisaría la de un compañero. En **máximo una página**:
a. ¿Qué partes **no aplican** a Fracttal tal como se describe el contexto, y por qué?

b. ¿Qué está **directamente mal** o se va a romper en producción? Diga qué pasaría y cuándo.

c. ¿Qué **falta** que usted consideraría imprescindible antes de aprobarla?

d. De todo lo anterior, ¿qué **rescataría** y por qué?

Puede acompañarlo de un diagrama si le ayuda a explicarse (Mermaid, imagen o foto de un papel, da igual). No hay una respuesta correcta única: hay respuestas que se sostienen y respuestas que se caen a la primera repregunta.

Entrega

1. El código y las evidencias, en un repositorio o zip que podamos ejecutar sin preguntarle nada.

2. Un reporte de máximo dos páginas con:

- Los puntos 2 y 3.
- Qué problemas de datos encontró en el punto 1, cómo los detectó y qué decidió hacer con cada uno. **Si sospecha que quedó alguno sin encontrar, dígalo: decirlo puntúa más que ocultarlo.**
- **Cómo usó la IA.** Damos por sentado que la usó. Queremos el detalle real: para qué exactamente ("le pasé la respuesta de la API y me generó el parser; el mapeo de estados lo corregí a mano", no "usé IA para el proyecto"), **qué le generó mal y cómo lo detectó** —si la respuesta es "nada", asumimos que no lo verificó— y qué decidió no delegarle.

3. Un archivo de contexto en el repo (`AGENTS.md`, `CLAUDE.md` o `README.md`, como quiera llamarlo) que le permita a un agente de IA, **sin usted presente**, levantar el mock, ejecutar una corrida completa y **distinguir un resultado correcto de uno incorrecto**. Pruébelo en una sesión nueva y limpia antes de entregar: vamos a ejecutarlo nosotros.
Ponderación: 60 % los tres puntos (20 % cada uno) · 40 % la entrega, el reporte y el archivo de contexto.

Después de la entrega habrá una sesión de 45 minutos en la que le pediremos ejecutar su proceso en vivo y comentaremos lo que entregó.

Reglas: no use datos reales de ninguna empresa; no pegue credenciales ni información de clientes en prompts de IA; si tiene dudas, escríbanos — preguntar no resta.

Entrega: envíe el repositorio o el zip a «CORREO» antes del «FECHA Y HORA». Cualquier duda, a esa misma dirección.